CSED233 · 데이터구조 · 포스텍

Ch7. Priority Queue & Heap

1. Priority Queue (PQ): 동기

Queue에 우선순위 추가

- **일반 Queue(큐)**: FIFO — 들어온 순서대로 나감
- **Priority Queue(우선순위 큐)**: 각 원소에 **priority(key)** 부여 → priority 순으로 나감
- Priority는 **unique일 필요 없음**
- 핵심 연산:
  - $\mathrm{Insert}(x, H)$: 임의 원소 삽입
  - $\mathrm{DeleteMin}(H)$: 최우선 원소 꺼내기 = $\mathrm{Min}(H) + \mathrm{Delete}$

왜 필요한가

- 응급실 환자 분류, OS 프로세스 스케줄링, Dijkstra 최단경로, 이벤트 시뮬레이션 등
- **현실은 FIFO보다 우선순위 기반 처리가 더 자연스러움**

2. 단순 구현의 한계

3가지 naive 구현 비교

구현	Insert	DeleteMin	이유
Unsorted linked list	$O(1)$	$O(n)$	삽입은 head, 삭제는 전체 스캔
Sorted linked list	$O(n)$	$O(1)$	삽입 위치 탐색, 삭제는 head
Heap(힙)	$O(\log n)$	$O(\log n)$	**균형**

Insight

- Unsorted/Sorted 모두 **한 쪽은** $O(n)$
- → **둘 다** $O(\log n)$ 달성이 Heap의 의미
- Insert와 DeleteMin이 **자주 번갈아 일어나는 상황**에선 heap이 최적

3. Binary Heap: 정의

두 가지 Property

Min-Heap은 다음 두 조건을 만족하는 tree:

1. **Structural property**: **Complete Binary Tree (CBT)**
   - → array로 pointer 없이 저장 가능 (Ch5 numbering 활용)
2. **Heap-order property**: **Min-tree**
   - $p(\text{parent}) \leq p(\text{child})$ for all nodes
   - → root가 전체 최솟값

> **Max-Heap**: (2')만 부등호 반대 $p(\text{parent}) \geq p(\text{child})$

CBT(완전 이진 트리) 선택 이유

- Heap의 모든 연산은 결국 트리 '높이'만큼 이동 → **높이 최소화가 관건**
- CBT는 n개 노드로 최소 높이 $\lfloor \log_2 n \rfloor$ 보장
- 또한 array 저장으로 pointer 오버헤드 제거 (cache friendly)

Heap ≠ BST(이진 탐색 트리)

- Heap은 **같은 level 내 순서에 제약 없음** (left/right 구분 의미 없음)
- BST는 left < root < right 이지만, heap은 parent ≤ children만 요구

4. Insert: Up-Heap Bubbling

2단계 알고리즘

1. **Hole 생성**: CBT 구조 유지 위해 **최하위 level의 가장 오른쪽**에 새 노드 slot 추가 → array 끝에 append
2. **Up-heap bubbling**: heap-order 회복을 위해 새 원소를 parent와 비교하며 **위로 swap**

시간복잡도 유도

- Bubbling은 **tree 높이만큼만** 위로 올라감
- CBT 높이 $= O(\log n)$
- 각 swap은 $O(1)$
- → **Insert =** $O(\log n)$

5. DeleteMin: Down-Heap Bubbling

2단계 알고리즘

1. **Root 제거 & 마지막 원소 이동**: root 삭제 → 최하위 최우측 원소를 root 자리로
2. **Down-heap bubbling**: heap-order 회복을 위해 **더 작은 자식**과 비교하며 아래로 swap

왜 '더 작은' 자식과 비교?

- 새 parent 자리에 오는 값이 **두 자식 중 작은 쪽**이어야 heap-order 유지
- 큰 자식과 swap하면 다른 쪽 자식 조건 위반
- → **반드시 min(left, right)과 비교** 후 swap

시간복잡도

- 트리 높이만큼 내려감 → $O(\log n)$

6. Quiz 정리

Q1. Min-heap의 heap-order property의 정확한 정의는?

> 힌트

Heap의 조건은 structural(모양)과 heap-order(값 관계) 두 가지다. 어느 쪽을 묻는지 구분하자.

> **정답**

개념은 힌트 참고.

- A. **Every parent node has value ≤ its children** ✓
- B. 모든 level이 꽉 참 → X (structural property와 혼동, 이건 full의 정의)
- C. Root가 최댓값 → X (min-heap은 root가 최솟값)
- D. Insertion은 root에서 → X (insertion은 맨 끝에서)

정답: A — Every parent node has value ≤ its children.

Q2. 왜 $O(\log n)$이 naive들보다 나은가?

> **힌트**

Insert와 DeleteMin이 번갈아 빈번하게 일어나는 워크로드를 생각하자.

> **정답**

개념은 힌트 참고.

- Insert와 DeleteMin이 **둘 다** $O(\log n)$ → 둘 다 빈번할 때 최적
- n번 연속 insert+delete 시: heap $O(n \log n)$ vs naive $O(n^2)$

7. Heap Building: n개 원소로 heap 만들기

두 방법

방법 1 (Obvious): 빈 heap에 하나씩 insert

- $\sum_{i=1}^{n} O(\log i) = O(n \log n)$

방법 2 (Heapify): 임의 순서로 CBT 만든 뒤 한 번에 정리

- $O(n)$ (증명은 § 8)

Heapify 알고리즘 (Bottom-up)

```
입력: array-based CBT (n 노드)
1) k = floor(n/2)  ← 마지막 non-terminal node index
2) for k downto 1:
     if subtree(k)가 heap order 위반:
        down-heap bubble at k
```

- **핵심**: **밑에서 위로**, 각 서브트리를 heap으로 만들면서 올라감
- 이미 자식이 heap인 상태에서 root 하나만 내려보내면 됨 → down-heap 활용

왜 마지막 non-leaf 부터?

- **Leaf 노드는 자기 자신이 이미 heap** (자식 없음) → 건드릴 필요 없음
- 마지막 non-leaf = 가장 큰 index의 parent = $\lfloor n/2 \rfloor$
- Top-down으로 하면 각 노드의 이동이 더 길어짐 → bottom-up이 더 효율적

8. Heapify Time Complexity: $O(n)$ 증명

핵심 아이디어

- Level 0 (root)는 높이 h, level h (leaf)는 높이 0
- **Level k 서브트리 개수** $\leq 2^k$
- **Level k 서브트리 높이** $\leq h - k$
- Down-heap의 시간 = 서브트리 높이

수식 유도

$$\text{Total} \leq \sum_{k=0}^{h} 2^k \cdot (h - k)$$

치환 $j = h - k$:

$$= \sum_{j=0}^{h} 2^{h-j} \cdot j = 2^h \sum_{j=0}^{h} \frac{j}{2^j}$$

$\sum_{j=0}^{\infty} \frac{j}{2^j} = 2$ (등비수열 도함수 공식):

$$\leq 2^h \cdot 2 = 2^{h+1}$$

$n \approx 2^{h+1} - 1$ (정확히는 perfect tree의 등식; CBT는 2^h ≤ n < 2^(h+1)을 만족하므로 2^(h+1) ≤ 2n) 이므로:

$$\text{Total} = O(2^{h+1}) = O(n)$$

교훈

Obvious insert 방법 ($O(n \log n)$)과 다르게 heapify는 **선형 시간** ($O(n)$).
이유: 낮은 level 노드들이 훨씬 많고 각자 이동 거리는 짧음 → 평균적으로 작음.

Summary

연산	시간복잡도	방법
Insert	$O(\log n)$	Append → Up-heap bubble
DeleteMin	$O(\log n)$	Root 제거 → last 원소 이동 → Down-heap bubble
BuildHeap (one-by-one)	$O(n \log n)$	n번 insert
Heapify (batch)	$O(n)$	Bottom-up down-heap
Find-Min	$O(1)$	Root 반환

- **Heap = CBT + heap-order** → array 저장, pointer 불필요
- Binary heap이 가장 흔함. Binomial/Fibonacci heap은 특수 상황